SRS — Sistema Web de Reconocimiento de Gestos de Mano

Sep 30, 2026 · @Sebastian

1. Introducción

GestoWeb (nombre provisional) es una aplicación web que reconoce 7 gestos de la mano con la cámara del computador y los convierte en acciones dentro de la página. Este documento define qué debe hacer, cómo se organiza y quién construye cada parte.

1.1 Propósito

Ser el acuerdo común del equipo (Terry, Juan, German y Neils) y del docente: qué se construye, qué queda fuera y cómo se comprueba que está terminado.

1.2 Alcance

El sistema incluye:

- Registro e inicio de sesión de usuarios.
- Reconocimiento de 7 gestos en tiempo real con la cámara web, procesado dentro del navegador.
- Una acción por gesto (por ejemplo, pasar de imagen o pausar un video) en un panel de demostración.
- Historial de gestos detectados y estadísticas por usuario.
- Panel de administración del catálogo de gestos y acciones.

El sistema no incluye:

- Lenguaje de señas completo ni reconocimiento de frases.
- Entrenar un modelo de inteligencia artificial propio: se usa uno ya entrenado.
- Aplicación móvil nativa.
- Guardar fotos o video del usuario.

1.3 Definiciones

Término	Significado
Gesto	Postura de la mano que el sistema sabe reconocer, como el pulgar arriba.

Término	Significado
Detección	Cada vez que el sistema reconoce un gesto, con fecha, hora y confianza.
Confianza	Qué tan seguro está el modelo de su respuesta, de 0 % a 100 %.
Umbral	Confianza mínima para aceptar un gesto como válido. Por defecto, 70 %.
Acción	Lo que ocurre en la página cuando se reconoce un gesto.
Sesión de reconocimiento	El tiempo entre encender y apagar la cámara.
Modelo	Programa ya entrenado que mira la imagen y dice qué gesto ve.
Arquitectura hexagonal	Forma de organizar el código donde la lógica central no depende de la base de datos, la web ni la cámara. Ver sección 7.
Puerto	Contrato que dice qué necesita la lógica central, sin decir cómo se hace.
Adaptador	Pieza concreta que cumple un puerto, por ejemplo, el que guarda datos en PostgreSQL.
SRS	Especificación de Requisitos de Software (en español, ERS).

2. Descripción general

GestoWeb reconoce los gestos dentro del navegador: la imagen de la cámara nunca sale del computador del usuario. Al servidor solo viaja el resultado: qué gesto, con qué confianza y a qué hora.

2.1 Cómo funciona, en pocas palabras

1. El usuario inicia sesión y abre la pantalla Reconocer.
2. El navegador pide permiso para usar la cámara.
3. El modelo mira la imagen varias veces por segundo y dice qué gesto ve.
4. Si el mismo gesto se mantiene estable un momento, el sistema lo acepta.
5. La página muestra el nombre del gesto y ejecuta la acción asignada.
6. El resultado se guarda en el historial del usuario.

2.2 Tipos de usuario

Tipo	Qué puede hacer
Visitante	Ver la página de inicio, registrarse e iniciar sesión.
Usuario	Reconocer gestos, cambiar qué acción hace cada gesto, ver su historial y sus estadísticas.
Administrador	Todo lo del usuario, más gestionar gestos, acciones y usuarios, y ver estadísticas generales.

2.3 Restricciones

- Debe funcionar en navegadores de escritorio actuales (Chrome, Edge, Firefox) con cámara web.
- La cámara solo se activa en sitios seguros: https o localhost.
- El backend usa arquitectura hexagonal; el frontend también aísla la cámara y el modelo detrás de un puerto.
- Stack de la materia: Angular, Node.js con Express y PostgreSQL.
- Equipo de 4 personas y un plazo estimado de 7 semanas.

2.4 Supuestos y dependencias

- Se usa el reconocedor de gestos de MediaPipe (Google): ya viene entrenado y es gratuito.
- El usuario tiene buena luz y la mano completa a la vista de la cámara.
- La versión 1 reconoce una mano a la vez.
- El archivo del modelo se guarda junto al frontend, para no depender de servidores externos durante la demo.

3. Catálogo de gestos

El sistema reconoce 7 gestos: son los que el modelo de MediaPipe trae ya entrenados, así que no hay que entrenar nada (fuente). Cada gesto dispara una acción en el panel de demostración, que es un visor de imágenes tipo presentación.

#	Gesto	Código del modelo	Cómo se hace	Acción por defecto
1	Palma abierta	Open_Palm	Mano abierta, los cinco dedos extendidos	Reproducir o pausar el pase automático
2	Puño cerrado	Closed_Fist	Todos los dedos cerrados	Detener y volver a la primera imagen
3	Índice arriba	Pointing_Up	Solo el índice extendido hacia arriba	Siguiente imagen
4	Victoria	Victory	Índice y medio abiertos en V	Imagen anterior
5	Pulgar arriba	Thumb_Up	Puño con el pulgar hacia arriba	Marcar la imagen como favorita
6	Pulgar abajo	Thumb_Down	Puño con el pulgar hacia abajo	Quitar la imagen de favoritas
7	Te quiero	ILoveYou	Pulgar, índice y meñique extendidos	Mostrar una reacción de corazón en pantalla

El modelo también responde Ninguno (None) cuando no ve un gesto conocido: en ese caso no pasa nada y no se guarda.

3.1 Reglas para aceptar un gesto

- **Confianza mínima:** 70 % por defecto. El administrador puede cambiarla por gesto.
- **Estabilidad:** el mismo gesto debe mantenerse medio segundo seguido. Evita acciones por movimientos de paso.
- **Pausa entre acciones:** después de ejecutar una acción, el mismo gesto no se repite durante 1 segundo.
- **Personalización:** cada usuario puede cambiar qué acción hace cada gesto; la tabla de arriba es la configuración inicial.

4. Requisitos funcionales

Son 22 requisitos en 5 módulos. Los 9 de prioridad alta son el mínimo que debe funcionar en la entrega; media es importante y baja se hace si sobra tiempo.

ID	El sistema debe…	Módulo	Prioridad
RF-01	Permitir que un visitante cree una cuenta con nombre, correo y contraseña.	Usuarios	Alta
RF-02	Permitir iniciar sesión con correo y contraseña, y mantener la sesión abierta de forma segura.	Usuarios	Alta
RF-03	Permitir cerrar sesión.	Usuarios	Alta
RF-04	Manejar dos roles, usuario y administrador, y mostrar cada pantalla solo al rol permitido.	Usuarios	Alta
RF-05	Permitir editar el nombre y cambiar la contraseña.	Usuarios	Baja
RF-06	Encender y apagar la cámara con un botón, pidiendo permiso al usuario.	Reconocimiento	Alta
RF-07	Mostrar el video en vivo con los puntos de la mano dibujados encima.	Reconocimiento	Media
RF-08	Reconocer los 7 gestos y mostrar en pantalla el nombre y la confianza.	Reconocimiento	Alta
RF-09	Aplicar las reglas de aceptación: confianza mínima, estabilidad y pausa entre acciones.	Reconocimiento	Alta
RF-10	Avisar con un mensaje claro si no hay cámara, si se negó el permiso o si no se ve la mano.	Reconocimiento	Media
RF-11	Ejecutar en el panel de demostración la acción asignada al gesto aceptado.	Acciones	Alta
RF-12	Permitir que el usuario cambie la acción de cada gesto y vuelva a la configuración inicial.	Acciones	Media
RF-13	Mostrar una guía visual de los 7 gestos y cómo hacerlos.	Acciones	Media
RF-14	Guardar cada gesto aceptado (gesto, confianza, mano, acción, fecha y hora) dentro de una sesión de reconocimiento.	Historial	Alta
RF-15	Mostrar el historial propio con filtros por fecha y por gesto.	Historial	Media
RF-16	Mostrar estadísticas propias: gesto más usado, total por gesto, confianza promedio y uso por día.	Historial	Media

ID	El sistema debe…	Módulo	Prioridad
RF-17	Exportar el historial propio a un archivo CSV.	Historial	Baja
RF-18	Permitir al administrador activar o desactivar gestos y editar su nombre, descripción y confianza mínima.	Administración	Media
RF-19	Permitir al administrador gestionar el catálogo de acciones disponibles.	Administración	Baja
RF-20	Permitir al administrador ver usuarios, cambiar su rol y activarlos o desactivarlos.	Administración	Media
RF-21	Mostrar al administrador estadísticas de todo el sistema.	Administración	Baja
RF-22	Guardar en una bitácora cada cambio que hace un administrador.	Administración	Baja

5. Requisitos no funcionales

Estos requisitos dicen qué tan bien debe funcionar el sistema; cada uno tiene una meta que se puede medir en las pruebas.

ID	Categoría	Requisito
RNF-01	Privacidad	La imagen de la cámara se procesa solo en el navegador. Nunca se envía ni se guarda.
RNF-02	Rendimiento	El reconocimiento corre a 15 imágenes por segundo o más en una laptop común.
RNF-03	Rendimiento	Entre hacer el gesto y ver la acción pasa 1 segundo o menos, incluido el medio segundo de estabilidad.
RNF-04	Rendimiento	El servidor responde en menos de medio segundo al guardar o consultar datos.
RNF-05	Precisión	Con buena luz, el gesto correcto se reconoce en 8 de cada 10 intentos o más.
RNF-06	Seguridad	Las contraseñas se guardan cifradas con bcrypt, nunca en texto plano.
RNF-07	Seguridad	Las pantallas privadas exigen un token de sesión (JWT) que vence a las 8 horas.

ID	Categoría	Requisito
RNF-08	Seguridad	El sitio publicado usa https.
RNF-09	Usabilidad	Un usuario nuevo reconoce su primer gesto en menos de 2 minutos, sin ayuda.
RNF-10	Usabilidad	Toda la interfaz está en español y se adapta a pantallas de laptop y tablet.
RNF-11	Compatibilidad	Funciona en las versiones actuales de Chrome, Edge y Firefox de escritorio.
RNF-12	Mantenibilidad	El núcleo no depende de Express, PostgreSQL ni MediaPipe: cambiar uno de ellos solo toca su adaptador.
RNF-13	Mantenibilidad	Todo el código está en TypeScript, en un repositorio Git con una rama por tarea y revisión antes de unir.
RNF-14	Pruebas	Cada caso de uso del núcleo tiene al menos una prueba automática.
RNF-15	Disponibilidad	El sistema queda publicado en un servicio gratuito en la nube para la demostración.

6. Casos de uso

El sistema tiene 7 casos de uso; el más importante es CU-03, Reconocer gestos en vivo, y se detalla paso a paso.

ID	Caso de uso	Actor	Requisitos que cubre
CU-01	Registrarse y editar su perfil	Visitante, Usuario	RF-01, RF-05
CU-02	Iniciar y cerrar sesión	Visitante, Usuario	RF-02, RF-03, RF-04
CU-03	Reconocer gestos en vivo	Usuario	RF-06 a RF-11, RF-14
CU-04	Personalizar acciones	Usuario	RF-12, RF-13
CU-05	Consultar historial y estadísticas	Usuario	RF-15, RF-16, RF-17
CU-06	Administrar gestos y acciones	Administrador	RF-18, RF-19, RF-22
CU-07	Administrar usuarios	Administrador	RF-20, RF-21, RF-22

6.1 CU-03 Reconocer gestos en vivo

Antes de empezar: el usuario inició sesión y tiene una cámara conectada.

Flujo normal

1. El usuario abre Reconocer y pulsa Encender cámara.
2. El navegador pide permiso y el usuario acepta.
3. El sistema carga el modelo y abre una sesión de reconocimiento en el servidor.
4. El sistema muestra el video con los puntos de la mano dibujados.
5. El usuario hace un gesto.
6. El sistema lo reconoce y revisa la confianza y la estabilidad; el gesto se acepta.
7. El panel ejecuta la acción asignada y muestra el nombre del gesto.
8. El sistema envía la detección al servidor, que la guarda.
9. El usuario pulsa Apagar cámara y el sistema cierra la sesión de reconocimiento.

Flujos alternativos

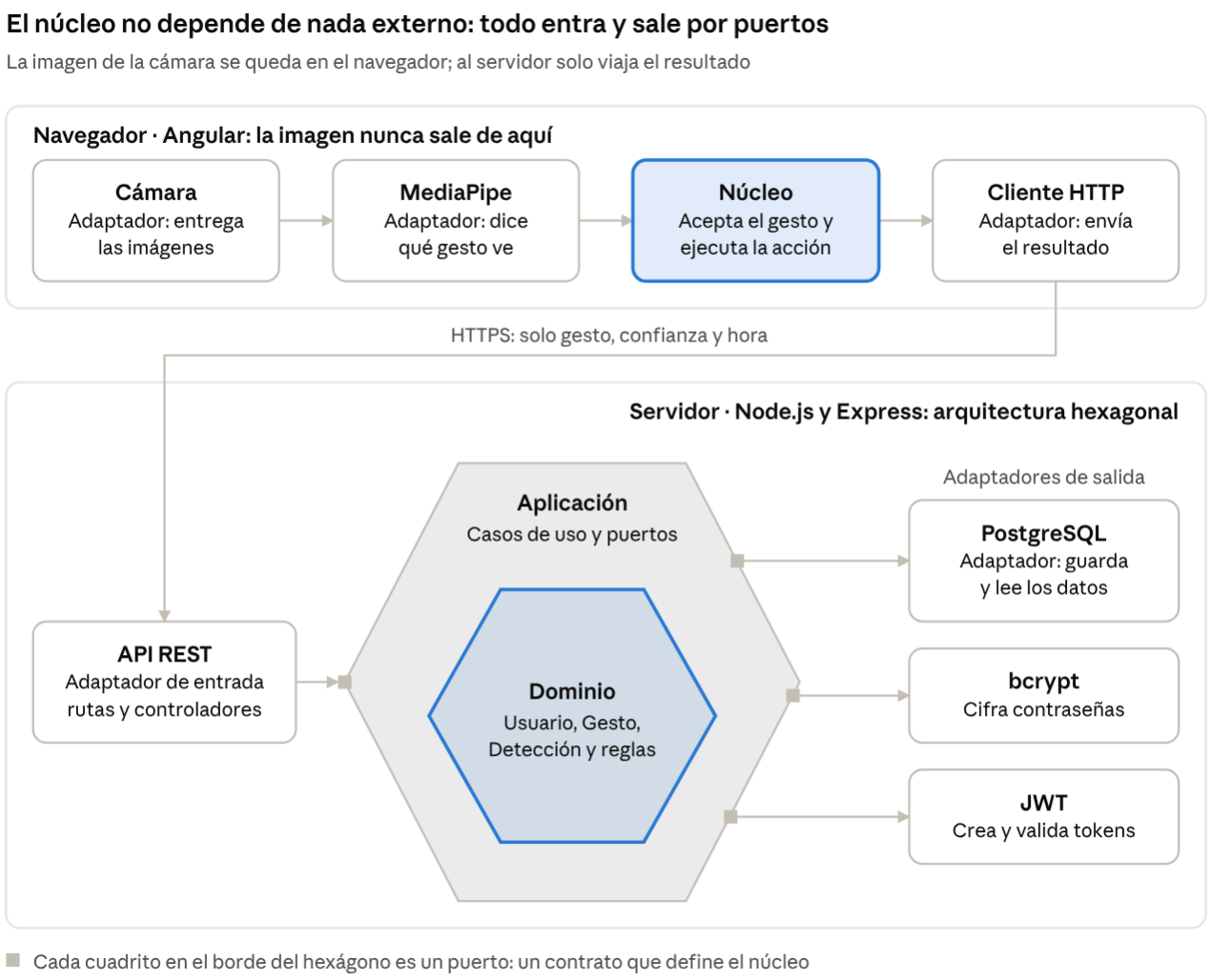
- **2a. Permiso negado:** se muestra cómo activar la cámara en el navegador; no se abre sesión.
- **6a. Confianza baja o gesto inestable:** se muestra Buscando gesto y no se ejecuta nada.
- **8a. El servidor no responde:** el reconocimiento sigue funcionando y el envío se reintenta.
- **9a. Se cierra la pestaña sin apagar la cámara:** la sesión abierta se cierra cuando el usuario inicia la siguiente.

Al terminar: las detecciones quedan en el historial del usuario.

7. Arquitectura hexagonal

La lógica del sistema vive en un núcleo que no conoce Express, PostgreSQL ni la cámara. Todo lo externo se conecta por puertos y adaptadores.

Piensen en un tomacorriente: el núcleo define la forma del enchufe (el puerto) y cualquier aparato con esa forma (el adaptador) se puede conectar. Si mañana cambian PostgreSQL por MySQL, solo se reemplaza el adaptador; el núcleo no se toca.



arquitectura de GestoWeb · navegador y servidor hexagonal

Las flechas muestran quién llama a quién. El núcleo solo conoce los puertos, nunca a Express, PostgreSQL, bcrypt, JWT ni MediaPipe.

7.1 Las tres capas

Las dependencias siempre apuntan hacia adentro: infraestructura conoce a aplicación, aplicación conoce a dominio, y dominio no conoce a nadie.

Capa	Qué contiene	Ejemplo en GestoWeb	Puede usar
Dominio	Entidades y reglas del negocio, sin librerías externas	Usuario, Gesto, Detección; la confianza debe estar entre 0 y 1	Nada
Aplicación	Casos de uso y puertos (contratos)	Registrar detección, consultar estadísticas; puerto DeteccionRepositorio	Dominio

Capa	Qué contiene	Ejemplo en GestoWeb	Puede usar
Infraestructura	Adaptadores que hablan con el exterior	Controladores Express, repositorios PostgreSQL, bcrypt, JWT	Aplicación y dominio

7.2 Casos de uso del núcleo (backend)

Cada caso de uso es una pieza pequeña con una sola tarea. Son los puertos de entrada del hexágono.

Grupo	Casos de uso
Usuarios	RegistrarUsuario, IniciarSesion, EditarPerfil
Reconocimiento	AbrirSesionReconocimiento, RegistrarDeteccion, CerrarSesionReconocimiento
Acciones	ObtenerAsignaciones, CambiarAsignacion, RestaurarAsignaciones
Historial	ConsultarHistorial, ConsultarEstadisticas, ExportarHistorial
Administración	EditarGesto, GestionarAcciones, GestionarUsuarios, ConsultarEstadisticasGenerales

7.3 Puertos de salida y sus adaptadores

Puerto (contrato)	Para qué sirve	Adaptador de la versión 1
UsuarioRepositorio	Guardar y buscar usuarios	PostgreSQL
GestoRepositorio	Leer y editar el catálogo de gestos	PostgreSQL
AccionRepositorio	Leer y editar el catálogo de acciones	PostgreSQL
AsignacionRepositorio	Guardar qué acción hace cada gesto por usuario	PostgreSQL
SesionRepositorio	Abrir y cerrar sesiones de reconocimiento	PostgreSQL

Puerto (contrato)	Para qué sirve	Adaptador de la versión 1
DeteccionRepositorio	Guardar detecciones y calcular estadísticas	PostgreSQL
BitacoraRepositorio	Registrar cambios del administrador	PostgreSQL
CifradorContrasenas	Cifrar y comparar contraseñas	bcrypt
GeneradorTokens	Crear y validar tokens de sesión	JWT

7.4 El frontend también usa puertos

- **ReconocedorGestos:** recibe una imagen y devuelve gesto, confianza y mano. Adaptador: MediaPipe. Cambiar a otro modelo solo toca este adaptador.
- **Camara:** enciende, apaga y entrega imágenes. Adaptador: la cámara del navegador.
- **ApiGestoWeb:** habla con el backend. Adaptador: cliente HTTP de Angular.
- **Reglas de aceptación:** confianza mínima, estabilidad y pausa viven en el núcleo del frontend, sin depender de la cámara. Así se prueban con datos falsos.

8. Organización del proyecto

Todo vive en un solo repositorio con cuatro carpetas: documentación, base de datos, backend y frontend. Cada carpeta tiene un dueño principal (ver sección 11).

8.1 Estructura de carpetas

```
gestoweb/
├── README.md                    cómo instalar y ejecutar
├── docs/                        este SRS, diagramas, manual de usuario
├── base-datos/                  Terry
│   ├── 01_tablas.sql            creación de tablas y restricciones
│   ├── 02_funciones_triggers.sql
│   ├── 03_vistas.sql            consultas de estadísticas
│   └── 04_datos_iniciales.sql   roles, 7 gestos, acciones, admin
├── backend/                     Juan (núcleo) y Terry (persistencia)
│   ├── src/
│   │   ├── dominio/             entidades y reglas, sin librerías
│   │   │   ├── usuario/
│   │   │   ├── gesto/
│   │   │   ├── accion/
│   │   │   └── deteccion/
│   │   ├── aplicacion/
```

```
|   |   |   ├── puertos/
|   |   |   |   ├── entrada/     contratos de los casos de uso
|   |   |   |   └── salida/      repositorios, cifrador, tokens
|   |   |   └── casos-uso/
|   |   ├── infraestructura/
|   |   |   ├── http/            rutas, controladores, middlewares
|   |   |   ├── persistencia/    repositorios PostgreSQL
|   |   |   └── seguridad/       bcrypt y JWT
|   |   ├── config/              variables de entorno y conexión
|   |   └── main.ts              conecta cada adaptador con su puerto
|   └── pruebas/
└── frontend/                    German (reconocimiento) y Neils (pantallas)
    └── src/
        ├── app/
        |   ├── nucleo/
        |   |   ├── dominio/     Gesto, ResultadoReconocimiento
        |   |   ├── puertos/     ReconocedorGestos, Camara, ApiGestoWeb
        |   |   └── servicios/   reglas de aceptación, ejecutor de acciones
        |   ├── adaptadores/
        |   |   ├── mediapipe/   adaptador del modelo
        |   |   ├── camara/      adaptador de la cámara
        |   |   └── http/        cliente de la API, interceptor del token
        |   ├── paginas/
        |   |   ├── auth/        login y registro
        |   |   ├── reconocer/   cámara y panel de demostración
        |   |   ├── configuracion/ acción de cada gesto
        |   |   ├── historial/
        |   |   ├── estadisticas/
        |   |   └── admin/
        |   └── compartido/      componentes comunes y guards
        └── assets/modelos/      archivo del modelo de gestos
```

El archivo main.ts del backend es el único lugar donde se decide qué adaptador usa cada puerto. Por eso cambiar de base de datos o de librería no rompe el resto.

8.2 Contrato de la API

Este contrato se congela en la semana 1: así el frontend avanza con datos de prueba mientras el backend se construye.

Método	Ruta	Qué hace	Rol
POST	/api/auth/registro	Crear una cuenta	Visitante
POST	/api/auth/login	Iniciar sesión y recibir el token	Visitante

Método	Ruta	Qué hace	Rol
GET	/api/usuarios/yo	Ver mi perfil	Usuario
PUT	/api/usuarios/yo	Editar mi nombre o contraseña	Usuario
GET	/api/gestos	Listar los gestos activos	Usuario
GET	/api/acciones	Listar las acciones disponibles	Usuario
GET	/api/asignaciones	Ver qué acción hace cada gesto	Usuario
PUT	/api/asignaciones/:idGesto	Cambiar la acción de un gesto	Usuario
POST	/api/asignaciones/restaurar	Volver a la configuración inicial	Usuario
POST	/api/sesiones	Abrir una sesión de reconocimiento	Usuario
PATCH	/api/sesiones/:id/cerrar	Cerrar la sesión	Usuario
POST	/api/sesiones/:id/detecciones	Guardar una detección	Usuario
GET	/api/historial	Historial con filtros de fecha y gesto	Usuario
GET	/api/historial/csv	Descargar el historial en CSV	Usuario
GET	/api/estadisticas	Mis estadísticas	Usuario
PUT	/api/admin/gestos/:id	Editar o desactivar un gesto	Administrador
POST, PUT, DELETE	/api/admin/acciones	Gestionar acciones	Administrador
GET	/api/admin/usuarios	Listar usuarios	Administrador
PATCH	/api/admin/usuarios/:id	Cambiar rol o estado de un usuario	Administrador
GET	/api/admin/estadisticas	Estadísticas generales	Administrador
GET	/api/admin/bitacora	Ver la bitácora de cambios	Administrador

8.3 Reglas de trabajo en equipo

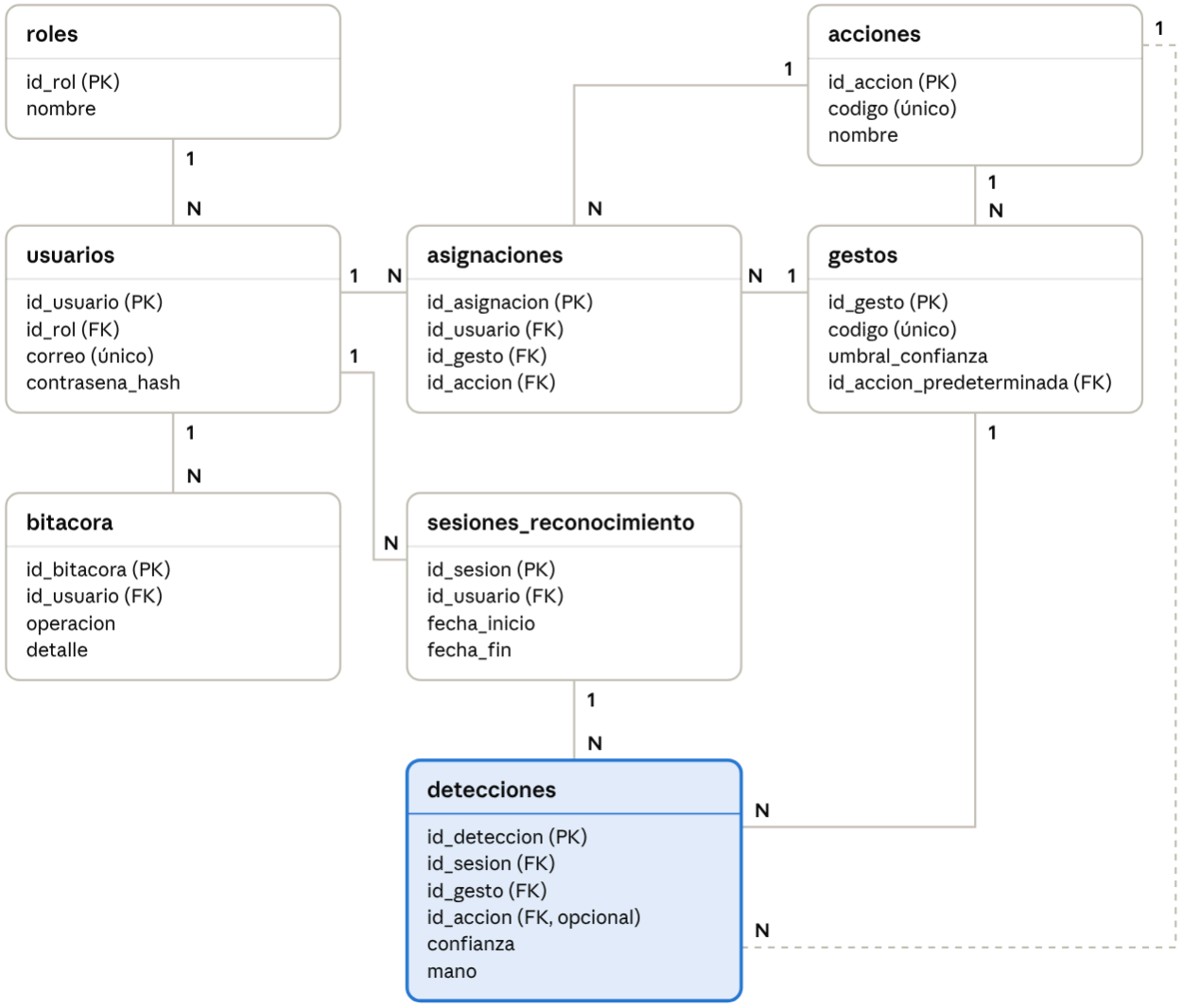
- Repositorio en GitHub con tres tipos de rama: main (lo que funciona), develop (integración) y una rama por tarea.
- Nadie sube directo a main ni a develop: todo entra por pull request revisado por otro integrante.
- Mensajes de commit cortos en español con prefijo: feat, fix o docs.
- Un archivo .env.example lista las variables; el .env real nunca se sube.
- Reunión corta dos veces por semana para revisar avances y bloqueos.

9. Base de datos

La base de datos usa PostgreSQL y tiene 8 tablas. La que más crece es detecciones: guarda una fila por cada gesto aceptado, nunca imágenes.

Usuarios y gestos son el centro; detecciones es la tabla que más crece

1 y N: una fila de un lado se relaciona con muchas del otro · punteada = relación opcional



modelo entidad-relación · 8 tablas

Cada detección pertenece a una sesión y cada sesión a un usuario: el historial de una persona se arma siguiendo esas dos líneas.

9.1 Resumen de tablas

Tabla	Qué guarda	Se relaciona con
roles	Los dos roles: USUARIO y ADMIN	usuarios
usuarios	Cuentas de las personas	roles, asignaciones, sesiones_reconocimiento, bitacora

Tabla	Qué guarda	Se relaciona con
acciones	Lo que puede pasar en el panel (siguiente, pausar…)	gestos, asignaciones, detecciones
gestos	Los 7 gestos, su confianza mínima y su acción por defecto	acciones, asignaciones, detecciones
asignaciones	Qué acción hace cada gesto para cada usuario	usuarios, gestos, acciones
sesiones_reconocimiento	Cada vez que alguien enciende y apaga la cámara	usuarios, detecciones
detecciones	Cada gesto aceptado, con confianza, mano y hora	sesiones_reconocimiento, gestos, acciones
bitacora	Cambios hechos por administradores	usuarios

9.2 Diccionario de datos

PK es la llave primaria (identifica la fila) y FK es la llave foránea (apunta a otra tabla).

roles

Campo	Tipo	Reglas	Para qué
id_rol	SERIAL	PK	Identificador
nombre	VARCHAR(20)	Único, obligatorio	USUARIO o ADMIN
descripcion	VARCHAR(150)	Opcional	Explicación del rol

usuarios

Campo	Tipo	Reglas	Para qué
id_usuario	SERIAL	PK	Identificador
nombre	VARCHAR(80)	Obligatorio	Nombre visible
correo	VARCHAR(120)	Único, obligatorio	Usuario para iniciar sesión
contrasena_hash	VARCHAR(100)	Obligatorio	Contraseña cifrada con bcrypt

Campo	Tipo	Reglas	Para qué
id_rol	INTEGER	FK a roles; por defecto USUARIO	Permisos
activo	BOOLEAN	Por defecto verdadero	Un usuario inactivo no puede entrar
fecha_registro	TIMESTAMP	Por defecto la hora actual	Cuándo se creó
ultimo_acceso	TIMESTAMP	Opcional	Último inicio de sesión

acciones

Campo	Tipo	Reglas	Para qué
id_accion	SERIAL	PK	Identificador
codigo	VARCHAR(30)	Único, obligatorio	Nombre que entiende el frontend, como SIGUIENTE
nombre	VARCHAR(60)	Obligatorio	Nombre visible, como Siguiente imagen
descripcion	VARCHAR(200)	Opcional	Qué hace
activa	BOOLEAN	Por defecto verdadero	Si se puede asignar

gestos

Campo	Tipo	Reglas	Para qué
id_gesto	SERIAL	PK	Identificador
codigo	VARCHAR(30)	Único, obligatorio	Etiqueta del modelo, como Thumb_Up
nombre	VARCHAR(50)	Obligatorio	Nombre visible, como Pulgar arriba
descripcion	VARCHAR(200)	Opcional	Cómo se hace el gesto
umbral_confianza	NUMERIC(3,2)	Entre 0 y 1; por defecto 0.70	Confianza mínima para aceptarlo

Campo	Tipo	Reglas	Para qué
id_accion_predeterminada	INTEGER	FK a acciones	Acción inicial para usuarios nuevos
activo	BOOLEAN	Por defecto verdadero	Si se reconoce o se ignora

asignaciones

Campo	Tipo	Reglas	Para qué
id_asignacion	SERIAL	PK	Identificador
id_usuario	INTEGER	FK a usuarios	Dueño de la configuración
id_gesto	INTEGER	FK a gestos; único junto con id_usuario	Gesto configurado
id_accion	INTEGER	FK a acciones	Acción elegida
fecha_actualizacion	TIMESTAMP	Por defecto la hora actual	Último cambio

sesiones_reconocimiento

Campo	Tipo	Reglas	Para qué
id_sesion	SERIAL	PK	Identificador
id_usuario	INTEGER	FK a usuarios	Quién usó la cámara
fecha_inicio	TIMESTAMP	Por defecto la hora actual	Cuándo se encendió
fecha_fin	TIMESTAMP	Vacía mientras está abierta	Cuándo se apagó
navegador	VARCHAR(100)	Opcional	Para revisar problemas de compatibilidad
total_detecciones	INTEGER	Por defecto 0	Lo actualiza un trigger

detecciones

Campo	Tipo	Reglas	Para qué
id_deteccion	BIGSERIAL	PK	Identificador
id_sesion	INTEGER	FK a sesiones_reconocimiento	Sesión a la que pertenece
id_gesto	INTEGER	FK a gestos	Gesto reconocido
id_accion	INTEGER	FK a acciones; opcional	Acción que se ejecutó
confianza	NUMERIC(4,3)	Entre 0 y 1	Seguridad del modelo
mano	VARCHAR(10)	Izquierda o Derecha	Mano usada
fecha_hora	TIMESTAMP	Por defecto la hora actual	Cuándo ocurrió

bitacora

Campo	Tipo	Reglas	Para qué
id_bitacora	BIGSERIAL	PK	Identificador
id_usuario	INTEGER	FK a usuarios	Administrador que hizo el cambio
tabla_afectada	VARCHAR(40)	Obligatorio	Dónde fue el cambio
operacion	VARCHAR(10)	INSERT, UPDATE o DELETE	Tipo de cambio
detalle	JSONB	Opcional	Valores antes y después
fecha_hora	TIMESTAMP	Por defecto la hora actual	Cuándo ocurrió

9.3 Lógica dentro de la base de datos

Estas piezas cubren los temas de la materia (triggers, funciones, procedimientos y vistas) y quitan trabajo repetido al backend.

Tipo	Nombre	Qué hace
Trigger	trg_asignaciones_iniciales	Al crear un usuario, le copia la acción por defecto de los 7 gestos.

Tipo	Nombre	Qué hace
Trigger	trg_contar_detecciones	Al guardar una detección, suma 1 al total de su sesión.
Trigger	trg_validar_sesion_abierta	Rechaza detecciones si la sesión ya está cerrada.
Procedimiento	sp_abrir_sesion	Cierra cualquier sesión abierta del usuario y abre una nueva.
Procedimiento	sp_restaurar_asignaciones	Devuelve al usuario la configuración inicial de acciones.
Función	fn_estadisticas_usuario	Total por gesto y confianza promedio de un usuario entre dos fechas.
Vista	v_historial	Detecciones con nombre del gesto, acción y usuario, lista para mostrar.
Vista	v_estadisticas_generales	Uso de cada gesto en todo el sistema, para el administrador.

La bitácora la escribe el backend en la misma transacción que el cambio, porque solo el backend sabe qué administrador lo hizo.

9.4 Índices y datos iniciales

- **Índices:** en detecciones por id_sesion y por fecha_hora, y en sesiones_reconocimiento por id_usuario. Aceleran el historial y las estadísticas.
- **Datos iniciales:** 2 roles, 8 acciones (las 7 de la sección 3 más Ninguna), 7 gestos con su acción por defecto y 1 administrador de prueba.

10. Tecnologías y herramientas

Se usa el stack de la materia más una sola pieza nueva: MediaPipe, que hace el reconocimiento dentro del navegador.

Parte	Herramienta	Para qué
Frontend	Angular con TypeScript	Pantallas, rutas, guards e interceptor del token
Reconocimiento	MediaPipe Gesture Recognizer, paquete @mediapipe/tasks-vision (guía web)	Reconocer los 7 gestos y los puntos de la mano en el navegador

Parte	Herramienta	Para qué
Gráficos	Chart.js	Estadísticas del usuario y del administrador
Backend	Node.js con Express y TypeScript	API REST y núcleo hexagonal
Base de datos	PostgreSQL con la librería pg	Guardar usuarios, configuración y detecciones
Seguridad	bcrypt y JWT	Cifrar contraseñas y manejar la sesión
Pruebas	Jest en el backend y las herramientas de prueba de Angular	Probar casos de uso y reglas de aceptación
API	Colección de Postman compartida	Probar endpoints sin el frontend
Equipo	Git, GitHub y GitHub Projects	Código, revisiones y tablero de tareas
Diagramas	draw.io	Diagramas de casos de uso y de la base de datos
Publicación	Railway para backend y base de datos; Vercel o Netlify para el frontend	Demo en línea con https; revisar los planes gratuitos vigentes

11. División del trabajo

Cada integrante es dueño de una parte del hexágono: Juan el núcleo del backend, Terry la base de datos, German el reconocimiento y Neils las pantallas. Como los contratos (puertos y API) se fijan en la semana 1, los cuatro avanzan en paralelo.

Integrante	Rol	Carpetas	Requisitos principales
Juan	Coordinador y núcleo del backend	backend/src/dominio, aplicacion, infraestructura/http y seguridad, main.ts	RF-01 a RF-05 y RF-14 en el servidor; casos de uso de la sección 7.2

Integrante	Rol	Carpetas	Requisitos principales
Terry	Base de datos y persistencia	base-datos/, backend/src/infraestructura/persistencia	Sección 9; datos de RF-14 a RF-17 y RF-22
German	Reconocimiento y acciones	frontend: nucleo, adaptadores/mediapipe y camara, paginas/reconocer	RF-06 a RF-11 y RF-13
Neils	Pantallas de usuario y administración	frontend: paginas de auth, configuracion, historial, estadisticas y admin; adaptadores/http; compartido	Pantallas de RF-01 a RF-05, RF-12 y RF-15 a RF-21

11.1 Juan: coordinador y núcleo del backend

- [ ] Crear el repositorio, las ramas y el tablero de tareas.
- [ ] Acordar con el equipo el contrato de la API y los puertos.
- [ ] Crear las entidades del dominio: Usuario, Gesto, Accion, Asignacion, SesionReconocimiento y Deteccion.
- [ ] Programar los casos de uso de usuarios, reconocimiento, acciones e historial.
- [ ] Programar rutas y controladores Express, y los middlewares de token y de rol.
- [ ] Programar los adaptadores de bcrypt y JWT.
- [ ] Armar main.ts, que conecta cada adaptador con su puerto.
- [ ] Escribir pruebas automáticas de los casos de uso.
- [ ] Publicar el backend y coordinar la integración final.

11.2 Terry: base de datos y persistencia

- [ ] Dibujar el diagrama entidad-relación final en draw.io.
- [ ] Escribir el script de tablas, llaves y restricciones.
- [ ] Programar los triggers, procedimientos, función y vistas de la sección 9.3.
- [ ] Cargar los datos iniciales: roles, acciones, gestos y administrador de prueba.
- [ ] Programar los repositorios PostgreSQL que cumplen los puertos de salida.
- [ ] Escribir las consultas de historial con filtros y de estadísticas.
- [ ] Registrar la bitácora de cambios del administrador.
- [ ] Cargar muchos datos de prueba para medir la velocidad de las consultas (RNF-04).
- [ ] Publicar la base de datos en la nube.

11.3 German: reconocimiento y acciones

- [ ] Definir los puertos ReconocedorGestos y Camara.
- [ ] Programar el adaptador de cámara: permiso, encender, apagar y mensajes de error.
- [ ] Programar el adaptador de MediaPipe: cargar el modelo y reconocer cada imagen.
- [ ] Dibujar los puntos de la mano sobre el video.
- [ ] Programar las reglas de aceptación con sus pruebas.
- [ ] Construir el panel de demostración con las 8 acciones.
- [ ] Armar la pantalla Reconocer: abrir sesión, enviar detecciones y cerrar sesión.
- [ ] Crear la guía visual de los 7 gestos.
- [ ] Medir precisión y velocidad (RNF-02, RNF-03 y RNF-05).

11.4 Neils: pantallas y administración

- [ ] Definir el diseño visual: colores, menú y plantilla de páginas.
- [ ] Programar el puerto ApiGestoWeb y su adaptador HTTP con el interceptor del token.
- [ ] Construir login, registro y perfil, con guards por rol.
- [ ] Construir la pantalla de configuración de acciones por gesto.
- [ ] Construir el historial con filtros y descarga en CSV.
- [ ] Construir las estadísticas con gráficos.
- [ ] Construir el panel de administración: gestos, acciones, usuarios, estadísticas y bitácora.
- [ ] Publicar el frontend.

11.5 Tareas de todos

- [ ] Documentar su parte en docs/ y en el README.
- [ ] Revisar al menos un pull request de otro integrante por semana.
- [ ] Preparar su parte de la presentación final.

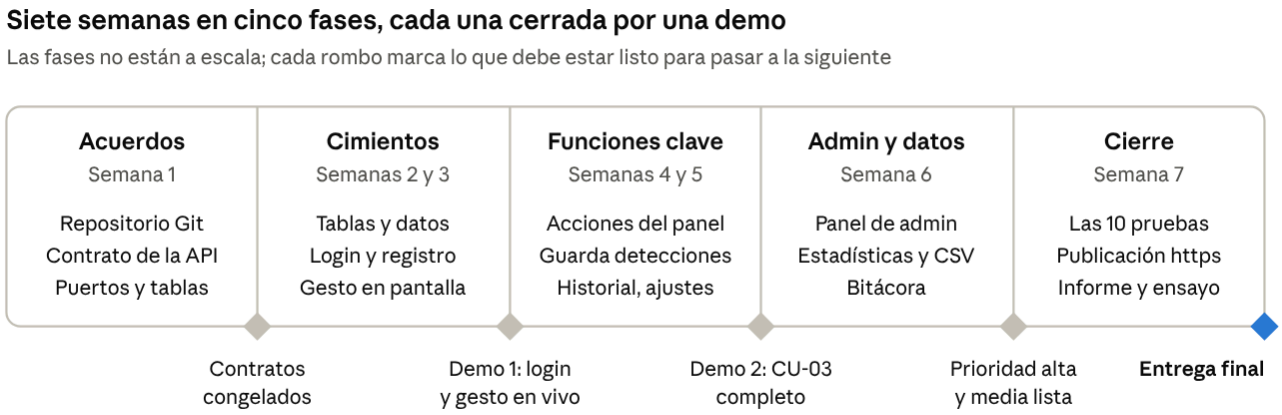
11.6 Quién necesita qué de quién

Quién	Necesita	De quién	Cuándo	Mientras tanto
German y Neils	Contrato de la API	Juan	Fin de la semana 1	Nada: es lo primero que se hace
Terry	Contrato de los puertos de salida	Juan	Fin de la semana 1	Avanza con el script de tablas

Quién	Necesita	De quién	Cuándo	Mientras tanto
Juan	Repositorios PostgreSQL	Terry	Semana 3	Usa repositorios en memoria
Neils	Endpoints funcionando	Juan y Terry	Semana 4	Usa datos simulados en el frontend
German	Endpoint de detecciones	Juan	Semana 4	Muestra las detecciones solo en pantalla

12. Plan por fases y pruebas

El proyecto se hace en 5 fases durante 7 semanas, y cada fase termina con una demo que prueba algo concreto. La entrega está lista cuando los 9 requisitos de prioridad alta funcionan en línea y pasan las pruebas P-01 a P-07.



plan por fases · 7 semanas, 5 hitos

Si una demo no pasa, la fase siguiente empieza igual, pero lo pendiente se corrige primero en la reunión de esa semana.

12.1 Pruebas de aceptación

Prueba	Cómo se hace	Se aprueba si	Requisito
P-01 Precisión	3 personas hacen cada gesto 10 veces con buena luz	8 de 10 aciertos o más por gesto	RF-08, RNF-05

Prueba	Cómo se hace	Se aprueba si	Requisito
P-02 Velocidad	Contador de imágenes por segundo, visible solo en modo desarrollo, en 2 laptops	15 por segundo o más	RNF-02
P-03 Respuesta	Grabar la pantalla y medir desde el gesto hasta la acción	1 segundo o menos	RNF-03
P-04 Privacidad	Revisar la pestaña Red del navegador durante 1 minuto de uso	Ninguna petición lleva imágenes	RNF-01
P-05 Permisos	Abrir una ruta de administrador sin token y con un usuario normal	El servidor rechaza ambos intentos	RF-04, RNF-07
P-06 Contraseñas	Revisar la tabla usuarios	Ninguna contraseña es legible	RNF-06
P-07 Base de datos	Crear un usuario; guardar una detección en una sesión cerrada	Se crean 7 asignaciones; la detección se rechaza	Sección 9.3
P-08 Usuario nuevo	Una persona fuera del equipo usa el sistema sin ayuda	Reconoce su primer gesto en menos de 2 minutos	RNF-09
P-09 Hexagonal	Cambiar en main.ts los repositorios PostgreSQL por unos en memoria y correr las pruebas	Las pruebas pasan sin tocar el núcleo	RNF-12
P-10 Navegadores	Hacer CU-03 completo en Chrome, Edge y Firefox	Funciona en los tres	RNF-11

13. Riesgos, limitaciones y trabajo futuro

El mayor riesgo es la demo en vivo: luz, cámara y permisos del navegador fallan más que el código, así que se prueban en el lugar de la presentación antes del día.

13.1 Riesgos

Riesgo	Efecto	Qué hacemos
Poca luz o fondo confuso	Baja la precisión	Consejos de luz en la guía, umbral ajustable y ensayo en el aula
Gestos parecidos, como índice arriba y victoria	Se ejecuta la acción equivocada	Regla de estabilidad, pausa entre acciones y confianza visible en pantalla
Laptop lenta	Menos imágenes por segundo	Usar la cámara a resolución baja y medir con P-02 desde la fase 1
La cámara no enciende en la demo	No hay presentación	Probar el sitio publicado con https y tener localhost como plan B
Un integrante se atrasa	Bloquea a otros	Contratos congelados en la semana 1, datos simulados y reunión dos veces por semana
Integrar todo al final	Muchos errores juntos	Unir a develop cada semana
Cambia la versión de MediaPipe	El modelo no carga	Fijar la versión del paquete y guardar el modelo en assets

13.2 Limitaciones de la versión 1

- Solo reconoce los 7 gestos del modelo.
- Reconoce una mano a la vez.
- Solo gestos quietos: no entiende movimientos como deslizar la mano.
- Depende de la calidad de la cámara y de la luz.

13.3 Trabajo futuro

- Entrenar gestos propios: MediaPipe permite añadir un clasificador personalizado.
- Reconocer gestos con movimiento, como deslizar a la izquierda o a la derecha.
- Controlar aplicaciones reales, como una presentación o un reproductor de música.
- Reconocer las dos manos a la vez.

Fuentes

- MediaPipe Gesture Recognizer: guía de la tarea
- MediaPipe Gesture Recognizer: guía para web y JavaScript